



AWS Cloud Practitioner

One-Day Hands-On Course

A hands-on introduction to Amazon Web Services

Aligned to: AWS Certified Cloud Practitioner (CLF-C02)

 **Today:** Five modules and eight hands-on labs in the AWS Management Console — roughly 70% of the day is at the keyboard. You will build a network, launch servers, deploy a web application, store objects, run serverless code, stand up a database, and optimize what it all costs.



Course Objectives

Upon completion you will be able to

- Explain the core concepts of cloud computing and the benefits of using AWS
- Identify and describe key AWS services, including EC2, S3, VPC, IAM, Lambda, RDS and more
- Understand the AWS global infrastructure and Regions
- Create and manage AWS resources using the AWS Management Console and CLI
- Implement basic security best practices on AWS
- Optimize AWS costs using various strategies
- Prepare for the AWS Certified Cloud Practitioner exam

Course modules

Module	Topic
1	Introduction to Cloud Computing
2	Core AWS Services — Security, Networking, Compute, Storage, Databases
3	AWS Global Infrastructure
4	Billing and Cost Optimization
5	Best Practices and Troubleshooting

 **Level:** Introductory. No prior AWS experience is assumed — only general familiarity with IT concepts (servers, storage, networks).



Course Flow — 70% Hands-On / 30% Theory

Morning

Welcome & environment check

Module 1: Introduction to Cloud Computing

Module 2A: Security, Identity & Compliance

 **Lab 1:** Configuring IAM Roles and Policies

Module 2B: Networking & Content Delivery

 **Lab 2:** Security Groups and Network ACLs

 Break

Module 2C: Compute — EC2

 **Lab 3:** Creating EC2 Instances

Module 2C: Deploying applications on EC2

 **Lab 4:** Deploying a Web Application on EC2

 Lunch

Afternoon

Module 2D: Storage — S3 & EBS

 **Lab 5:** Using S3 for Object Storage

Module 2C: Serverless — Lambda

 **Lab 6:** Serverless Functions with Lambda

Module 2E: Databases — RDS & DynamoDB

 **Lab 7A:** Launch the RDS instance

 Break (RDS provisions while you rest)

 **Lab 7B:** Connect to and explore RDS

Module 3: AWS Global Infrastructure

Module 4: Billing and Cost Optimization

 **Lab 8:** Cost Optimization Strategies & cleanup

Module 5: Best Practices, Troubleshooting & Exam Prep

Wrap-up

Lab conventions: everyone shares one AWS account in **us-east-2 (Ohio)**. Prefix every resource you create with your assigned username, e.g. `user7-web-sg`. Labs build on each other — keep resources until Lab 8's cleanup.



Module 1

Introduction to Cloud Computing



What Is Cloud Computing?

The on-demand delivery of IT resources over the internet with pay-as-you-go pricing.

- Instead of buying and maintaining physical data centers and servers, you rent compute, storage, databases and more from a cloud vendor (AWS, Azure, Google Cloud ...)
- Resources are provisioned in minutes via a web console, command line or API — and released just as fast
- Geographically distributed — deploy close to your users anywhere in the world

CapEx → OpEx

- **Capital expense:** large up-front hardware purchase you must forecast years ahead
- **Operational expense:** variable monthly cost that tracks actual usage

NIST's five essential characteristics

- **On-demand self-service** — provision without human interaction with the provider
- **Broad network access** — available over the network from standard clients
- **Resource pooling** — multi-tenant model; physical resources are shared and isolated by virtualization
- **Rapid elasticity** — scale out and in quickly, appearing unlimited
- **Measured service** — usage is metered, monitored and billed



Exam tip: Know these five characteristics — questions often describe one and ask you to name it.



Benefits of Cloud Computing — The Six Advantages

1. Trade fixed expense for variable expense

Pay only for what you consume instead of investing in data centers before you know how you'll use them.

2. Benefit from massive economies of scale

AWS aggregates usage from millions of customers and passes lower per-unit costs on.

3. Stop guessing capacity

Scale up or down in minutes — no more over-provisioning "just in case" or under-provisioning and losing customers.

4. Increase speed and agility

New resources are a click away; the cost of experimenting drops to nearly zero.

5. Stop spending money running data centers

Focus on your customers, not on racking, stacking and powering servers.

6. Go global in minutes

Deploy your application in multiple Regions around the world with a few clicks.

Why AWS specifically?

- Market leader with the broadest and deepest set of services (200+)
- Largest global footprint; mature security and compliance programs
- Everything is an API — automate all of it



Exam tip: The six advantages come straight from the AWS "Overview of Amazon Web Services" whitepaper and appear frequently in Domain 1 questions.



Types of Cloud Computing (Deployment Models)

Public cloud

All resources run on a provider's shared infrastructure (AWS). Fastest to start, pay-as-you-go, provider handles the hardware.

Private cloud

Cloud-style virtualization and self-service inside your own data center (VMware, OpenStack). You own capacity, patching and the CapEx.

Hybrid cloud

On-premises connected to the public cloud — VPN or Direct Connect, Storage Gateway, AWS Outposts. Common during migrations and for data-residency constraints.

Multi-cloud

Using two or more public clouds. Motivations: avoiding lock-in, best-of-breed services, acquisitions. Cost: more complexity and skills to maintain.

Migration strategies you will hear about (the "6 Rs")

- **Rehost** (lift and shift) · **Replatform** (lift, tinker and shift) · **Repurchase** (move to SaaS)
- **Refactor** (re-architect for cloud) · **Retire** · **Retain**

AWS hybrid tools: AWS Outposts (AWS racks in your building), Storage Gateway, Direct Connect, Site-to-Site VPN, Systems Manager for on-prem servers.

Key Concepts: IaaS, PaaS and SaaS

Model	What you manage	What the provider manages	AWS examples
IaaS Infrastructure as a Service	OS, runtime, middleware, application, data	Virtualization, servers, storage, networking	Amazon EC2, Amazon VPC, Amazon EBS
PaaS Platform as a Service	Application and data	Everything below, including OS patching and the runtime	AWS Elastic Beanstalk, Amazon RDS, AWS Lambda
SaaS Software as a Service	Your usage and configuration	The complete product	Amazon WorkMail, Amazon Chime, third-party apps in AWS Marketplace

	On-premises	IaaS (EC2)	PaaS (RDS / Beanstalk)	SaaS
Application	you	you	you	provider
Data	you	you	you	provider
Runtime	you	you	provider	provider
OS	you	you	provider	provider
Virtualization	you	provider	provider	provider
Hardware	you	provider	provider	provider
Networking	you	provider	provider	provider

Rule of thumb: the further right you move, the less you manage — and the less you can customize. **Managed** services (RDS, DynamoDB, Lambda) remove "undifferentiated heavy lifting"; **unmanaged** services (EC2) give you full control and full responsibility.



Primer: Where Do Things Live in AWS?

You will need these three terms for every lab today. Module 3 covers the global infrastructure in depth.

- **Region** — a geographic area with its own independent set of AWS resources, e.g. `us-east-2` (Ohio). *Today's labs all run in us-east-2.*
- **Availability Zone (AZ)** — one or more isolated data centers inside a Region (`us-east-2a`, `2b`, `2c`). Spreading across AZs is how you survive a data-center failure.
- **Edge location** — hundreds of small sites worldwide used by CloudFront and Route 53 to serve content close to users.

Scope matters: IAM and Route 53 are *global*. VPC, EC2, RDS and most others are *Regional* — pick the wrong Region in the console and your resources "disappear".

Three ways to talk to AWS

- **Management Console** — web UI; best for learning and one-off tasks
- **AWS CLI** — `aws s3 ls`; scriptable. **CloudShell** gives you a browser terminal with the CLI pre-installed
- **SDKs & Infrastructure as Code** — boto3, JavaScript, Java ...; CloudFormation, CDK, Terraform

```
Console  --+
CLI      --+--> HTTPS --> AWS API --> IAM authenticates
SDK/IaC  --+                and authorizes --> service acts
```

 **Exam tip:** Console = password (+ MFA). CLI/SDK = access key ID + secret access key — or better, an IAM role.



Module 1 Summary

- Cloud computing = on-demand IT resources over the internet, pay-as-you-go, CapEx → OpEx
- NIST's five characteristics: on-demand self-service, broad network access, resource pooling, rapid elasticity, measured service
- Six advantages of cloud computing
- Deployment models: public, private, hybrid, multi-cloud
- Service models: IaaS, PaaS, SaaS — managed vs unmanaged
- Regions, AZs, edge locations; console, CLI, SDK

Up next — Module 2A: before you touch any resource you need to understand who is allowed to do what. IAM first, then your first lab.



Module 2

Core AWS Services

2A Security & Identity · 2B Networking · 2C Compute · 2D Storage · 2E Databases



Module 2A

Security, Identity & Compliance — IAM and KMS

The Shared Responsibility Model

AWS: security *OF* the cloud

- Physical security of data centers, hardware and the global network
- Virtualization layer (hypervisor) and host operating systems
- The infrastructure under managed services (RDS, Lambda, S3 ...)
- Compliance of the infrastructure (SOC, ISO 27001, PCI DSS, HIPAA eligibility ...) — evidence in **AWS Artifact**

Customer: security *IN* the cloud

- Your data — classification, encryption at rest and in transit
- IAM users, groups, roles, permissions and MFA
- Guest OS patching and firewall configuration on EC2
- Application code, security groups, network ACLs, VPC design

Responsibility shifts with the service: on EC2 *you* patch the OS; on RDS AWS patches the engine but *you* control encryption, users and backup retention; on Lambda you own only your code and its permissions.

 **Exam tip:** "Who patches the OS on an EC2 instance?" → customer. "Who disposes of failed disks?" → AWS. Encryption is a *shared* control: AWS provides the capability, you turn it on.



AWS Identity and Access Management (IAM)

IAM controls *who* (authentication) can do *what* (authorization) on *which* resources.

- Global service — not tied to a Region; free of charge
- Integrated with every AWS service; managed via console, CLI or SDK

The root user

- Created with the account; signs in with the account email; **unrestricted** access that no policy can limit
- Use it only for tasks that require it (close the account, change support plan, some billing settings)
- **Enable MFA on root, lock the credentials away, create IAM identities for daily work**

Entity	What it is
User	A person or application with long-term credentials (password and/or access keys)
Group	A collection of users; attach policies once for everyone. Not nestable; a user may be in several groups
Role	An identity with <i>temporary</i> credentials that a trusted entity <i>assumes</i> — an EC2 instance, a Lambda function, a user in another account, a federated user
Policy	A JSON document that allows or denies actions on resources; attached to users, groups or roles

 **Exam tip:** An application on EC2 needs to read S3 → attach an **IAM role** to the instance. Never store access keys on the instance. (You create this role in Lab 1 and use it in Labs 3 and 5.)



IAM Policies and Least Privilege

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [ "s3:ListBucket", "s3:GetObject" ],
      "Resource": [
        "arn:aws:s3:::user7-bucket-4821",
        "arn:aws:s3:::user7-bucket-4821/*"
      ]
    }
  ]
}
```

- **Effect** — Allow or Deny · **Action** — the API calls (`service:Operation`)
- **Resource** — the ARN(s) it applies to · **Condition** (optional) — e.g. require MFA, restrict source IP

Evaluation rules

- Everything is **implicitly denied** by default
- An **explicit Allow** grants access ...
- ... but an **explicit Deny always wins**

Policy types

- **AWS managed** — maintained by AWS (`AmazonS3ReadOnlyAccess`, `AdministratorAccess`)
- **Customer managed** — you write and reuse across identities
- **Inline** — embedded in one identity

Principle of least privilege: grant only what the job needs. Start narrow; widen when a real need appears.

ARN: `arn:aws:iam::123456789012:role/user7-ec2-role`



Credentials, MFA and Key Management Service

Credentials

- **Password** → console (account ID or alias + user name)
- **Access key ID + secret** → CLI, SDK, API
- **Temporary credentials** (STS) → issued when a role is assumed; expire automatically
- **Password policy** — length, complexity, expiry, reuse

Multi-Factor Authentication

- Something you know + something you have
- Virtual (authenticator app), FIDO2 security key, hardware TOTP token
- Root first, then every human user

Beyond IAM users

- **IAM Identity Center** — single sign-on across many accounts; federate Active Directory, Okta, Entra ID (SAML 2.0)
- **Amazon Cognito** — sign-up / sign-in for *your application's* end users, including social logins

 Workforce → Identity Center. App customers → Cognito. Machines → IAM roles.

AWS Key Management Service (KMS)

- Create, rotate, disable and audit encryption keys; keys are Regional and identified by ARN
- Integrated with S3, EBS, RDS, DynamoDB, Lambda, Secrets Manager and 100+ services — encryption at rest is a checkbox
- Every use of a key is logged in CloudTrail
- **AWS managed keys** (free, per service) vs **customer managed keys** (you control policy and rotation)
- **AWS CloudHSM** — dedicated, single-tenant hardware security modules when regulations require it

S3 encryption options: SSE-S3 (S3 keys), SSE-KMS (KMS keys), SSE-C (your keys).
In transit: TLS via AWS Certificate Manager.



The Wider AWS Security Toolbox

AWS Shield

DDoS protection.
Standard free and always on; *Advanced* adds 24x7 response team and cost protection.

AWS WAF

Web application firewall — blocks SQL injection, cross-site scripting and bad IPs at CloudFront, ALB or API Gateway.

Amazon GuardDuty

Intelligent threat detection using CloudTrail, VPC Flow Logs and DNS logs.

Amazon Inspector

Automated vulnerability scanning of EC2, container images and Lambda.

Amazon Macie

Finds sensitive data (PII) in S3 with machine learning.

AWS Security Hub

Aggregates findings from GuardDuty, Inspector, Macie and partners against standards like CIS.

AWS Secrets Manager

Stores and rotates database passwords and API keys.

AWS Artifact

On-demand compliance reports (SOC, PCI, ISO) and agreements (BAA).

 **Exam tip:** match the keyword — "DDoS" → Shield · "SQL injection" → WAF · "compliance report" → Artifact · "PII in S3" → Macie · "threat detection" → GuardDuty · "vulnerabilities on EC2" → Inspector · "rotate DB password" → Secrets Manager.

Up next — Lab 1: create a group and a user, sign in as that user to see least privilege, and create the EC2 role your web server will use later today.



Time for Hands-On Practice!



Lab 1: Configuring IAM Roles and Policies



What You'll Build:

-  Sign in and set your Region to us-east-2
-  An IAM group with S3 read-only access, and a user in it
-  Sign in as that user and see least privilege in action
-  An IAM role for EC2 — used by your web server in Labs 3 and 5

Name everything `userN-...`



Lock It Down!



Module 2B

Networking & Content Delivery — VPC, Security Groups & NACLs, Route 53, CloudFront



Amazon VPC: Your Private Network in AWS

A logically isolated virtual network where you launch EC2 instances, RDS databases and other resources.

- Lives in one **Region** and spans all its AZs
- You choose a private IPv4 range in CIDR notation, e.g. **10.0.0.0/16** (65,536 addresses; /16 largest, /28 smallest); it cannot be changed later
- Every Region has a **default VPC** so you can launch instances immediately — real workloads get custom VPCs

Subnets

- A slice of the VPC range that lives in **exactly one AZ** (e.g. **10.0.1.0/24**); AWS reserves 5 IPs per subnet
- **Public subnet** — its route table sends **0.0.0.0/0** to an Internet Gateway
- **Private subnet** — no internet route; reaches out through a NAT Gateway

Component	Purpose
Internet Gateway	Two-way internet access for public subnets; one per VPC
Route table	Destination → target rules for subnet traffic; the <i>local</i> route is always present
NAT Gateway	Outbound-only internet for private subnets
Security group	Instance-level firewall
Network ACL	Subnet-level firewall
VPC Flow Logs	Capture IP traffic metadata for troubleshooting and security
Elastic IP	A static public IPv4 address you own and can re-assign
VPC endpoint	Private path to S3, DynamoDB and other services — no IGW needed



The Network You Will Build Today

```
+----- VPC  userN-vpc  10.0.0.0/16  (us-east-2) -----+
|
|  +-- Public subnet 1  10.0.0.0/20  (us-east-2a)  --+      +-- Public subnet 2  10.0.16.0/20  (us-east-2b)  --+  |
|  |  Network ACL: default (+ Deny rule, Lab 2)      |      |  Network ACL: default      |      | |
|  |  |                                              |      |  |
|  |  [ EC2 web server  (Lab 3 / 4) ]                |      |  [ EC2 web server #2  (Lab 4) ]      |      |
|  |  security group: userN-web-sg                    |      |  [ RDS MySQL  (Lab 7) ]      |      |
|  |  in: 80, 22 from 0.0.0.0/0    out: all            |      |  security group: userN-db-sg      |      |
|  |  |                                              |      |  in: 3306 from userN-web-sg only  |      |
|  +-----+-----+-----+-----+-----+-----+-----+
|
|          |
|  Route table (public):  10.0.0.0/16 -> local    0.0.0.0/0 -> igw  |
|          +-----+-----+-----+-----+-----+
|
|          |
|          [ Internet Gateway ]
|
+-----+-----+-----+-----+-----+-----+-----+
|
|          |
|          Internet  <--  your browser
```

Also today: S3 bucket (Lab 5) reached from EC2 via the IAM role (Lab 1) | Lambda function (Lab 6)

- Two subnets in two AZs — the minimum for a highly available design and a requirement for RDS
- The route to **0.0.0.0/0** via the IGW is what makes a subnet "public"; the instance also needs a public IP
- The database security group trusts the *web security group*, not an IP address — a core AWS pattern

Security Groups vs Network ACLs

	Security Group	Network ACL
Applies to	Instance / network interface	Subnet (every instance in it)
Rules	Allow only — anything not allowed is denied	Allow <i>and</i> Deny; numbered; evaluated lowest number first; * rule denies at the end
State	Stateful — return traffic is automatically allowed	Stateless — you must allow inbound <i>and</i> outbound explicitly (ephemeral ports 1024–65535!)
Defaults	Default SG: all outbound, inbound only from itself. New SG: no inbound	Default NACL: allow everything. New NACL: deny everything
Typical use	"Web servers accept 80/443 from anywhere; DB accepts 3306 only from the web SG"	Block a malicious IP range for a whole subnet; coarse subnet-level guardrails

Security group rule anatomy

- **Type / protocol** — HTTP (TCP 80), SSH (TCP 22), MySQL (TCP 3306), All ICMP ...
- **Source** — a CIDR (`0.0.0.0/0` = anywhere, `203.0.113.5/32` = one host) or *another security group ID*

 **Exam tip:** "Stateful", "instance level" → security group. "Stateless", "subnet level", "deny rule" → network ACL. Both are *your* responsibility.



Route 53, CloudFront and Hybrid Connectivity



Amazon Route 53 — DNS

- Highly available, scalable DNS with a **100% availability SLA**; also registers domains
- **Hosted zones** (public or private) hold records: A, AAAA, CNAME, MX, TXT, *Alias* (points at ELB, CloudFront, S3)
- **Routing policies:** simple · weighted (blue/green) · latency-based · failover (with health checks) · geolocation · geoproximity · multi-value



Amazon CloudFront — CDN

- Caches content at 600+ edge locations; origin can be S3, ELB, EC2 or any HTTP server
- Lower latency, lower origin load, lower data-transfer cost; HTTPS at the edge with free ACM certificates
- Integrates with Shield and WAF for edge protection; signed URLs for private content



Connecting networks

Service	Use when
VPC Peering	Two VPCs need private routing (not transitive)
Transit Gateway	Many VPCs and sites — hub-and-spoke
Site-to-Site VPN	Encrypted tunnel to on-premises over the internet; minutes to set up
Direct Connect	Dedicated private fiber — consistent bandwidth and latency; weeks to provision
VPC Endpoints	Reach S3/DynamoDB/other services without an IGW (PrivateLink)
Global Accelerator	Static anycast IPs routing users over the AWS backbone (Module 3)



"Private, dedicated, consistent" → Direct Connect. "Quick, encrypted, over the internet" → VPN. "Cache static content near users" → CloudFront. "DNS failover" → Route 53 health checks.

Up next — Lab 2: build the VPC in the diagram, create the web and database security groups, and add a Deny rule to the network ACL.



Time for Hands-On Practice!



Lab 2: Configuring Security Groups and Network ACLs



What You'll Build:

-  A VPC (10.0.0.0/16) with two public subnets in two AZs, an Internet Gateway and route table
-  `userN-web-sg` — HTTP and SSH from the internet
-  `userN-db-sg` — MySQL only from the web security group
-  A Deny rule on the network ACL — something a security group cannot do

Break follows • Labs 3–7 all run inside this VPC



Build the Network!



Module 2C

Compute — Amazon EC2

Amazon Elastic Compute Cloud (EC2)

Resizable virtual servers in the cloud — the IaaS workhorse of AWS. Launch in minutes, pay per second, stop or terminate any time.

What you choose at launch

- **AMI** (Amazon Machine Image) — the OS and pre-installed software template: Amazon Linux 2023, Ubuntu, Windows Server, Marketplace images, or your own custom AMIs
- **Instance type** — the CPU / memory / storage / network profile
- **Key pair** — public/private key for SSH (Linux) or password decryption (Windows); AWS keeps the public half, you download the `.pem` once
- **Network** — VPC, subnet, public IP, security group(s)
- **Storage** — root EBS volume size/type, extra volumes
- **IAM role** (instance profile), **user data** script, **tags**

Instance type families

Family	Optimized for	Examples
General purpose	Balanced; web servers, dev/test	t3, t4g (burstable), m6i, m7g
Compute optimized	High CPU; batch, gaming, HPC	c6i, c7g
Memory optimized	Large in-memory datasets; SAP, caches	r6i, x2, u-
Storage optimized	High local disk IOPS; NoSQL, data warehouses	i4i, d3
Accelerated	GPUs / ML chips	p5, g5, inf2, trn1

Naming: `t3.micro` = family *t*, generation 3, size *micro*.
Graviton (ARM) types end in *g* and cost ~20% less.

Instance lifecycle: pending → running → stopping → stopped → (start again) ... → terminated. Stopping keeps the EBS root volume; terminating deletes it by default. Public IPs change on stop/start unless you use an Elastic IP.



Connecting To and Securing an Instance

Ways in

- **EC2 Instance Connect** — browser-based SSH from the console; no key file to manage (Lab 3)
- **SSH with your key pair** — `ssh -i userN-key.pem ec2-user@<public-ip>`
- **Session Manager** (Systems Manager) — no open port 22 at all; audited sessions; the production best practice
- **RDP** — Windows instances, port 3389

Root vs instance store

- **EBS-backed** (the norm) — persistent network volume; survives stop/start; snapshots to S3
- **Instance store** — physically attached, very fast, *ephemeral* — data is lost on stop or termination

Security checklist for a web server

- Security group: only the ports you serve (80/443); SSH restricted to your IP or removed in favor of Session Manager
- IAM role instead of access keys on the box
- Patch the OS — it is *your* side of the Shared Responsibility Model (Systems Manager Patch Manager helps)
- Encrypt EBS volumes (KMS) — one checkbox, no performance penalty
- Tags: `Name`, `Owner`, `Environment`, `CostCenter` — used for cost allocation and IAM conditions
- Use IMDSv2 (token-based instance metadata) — the default on Amazon Linux 2023



Exam tip: "Data lost when instance stops" → instance store. "Connect with no inbound ports open" → Session Manager. "Static public IP that survives stop/start" → Elastic IP.

Up next — Lab 3: create a key pair, launch an Amazon Linux 2023 instance into your VPC with your security group and IAM role, and connect to it from the browser.



Time for Hands-On Practice!



Lab 3: Creating EC2 Instances



What You'll Build:

- ✓ A key pair `userN-key`
- ✓ A t3.micro Amazon Linux 2023 instance in your VPC with `userN-web-sg` and `userN-ec2-role`
- ✓ Connect from the browser with EC2 Instance Connect
- ✓ Prove the instance uses its IAM role — `aws s3 ls` with no access keys



Launch!



Deploying Applications on EC2

Three ways to get software onto an instance

- **Manually** — SSH in, install packages, copy files. Fine for learning; does not scale and drifts over time
- **User data** — a shell script that runs *once* at first boot. Installs and configures automatically; the same script gives you identical servers every time
- **Custom AMI** — bake the configured instance into an image, then launch copies in seconds. Combine with user data for the last-mile config

```
#!/bin/bash
dnf install -y httpd
systemctl enable --now httpd
echo "<h1>Hello from $(hostname -f)</h1>" \
> /var/www/html/index.html
```

From one server to a fleet

- **Elastic Load Balancing** — distributes traffic across instances in multiple AZs and health-checks them. *Application* LB (HTTP/HTTPS, path routing) · *Network* LB (TCP/UDP, ultra-low latency, static IP) · *Gateway* LB (third-party appliances)
- **Auto Scaling group** — launches from a *launch template* (AMI + type + user data), keeps the desired count healthy, scales on CloudWatch metrics (Module 4)
- **AWS Elastic Beanstalk** — upload code; it provisions EC2, ELB, Auto Scaling and monitoring for you (PaaS)
- **Containers** — Amazon ECS / EKS, with Fargate to run them serverless
- **AWS CodeDeploy / CodePipeline** — automated CI/CD onto EC2, Lambda or ECS

 **Exam tip:** "Just deploy my code, manage the infrastructure for me" → Elastic Beanstalk. "Layer 7 path-based routing" → ALB. "Run containers without managing servers" → Fargate.

Up next — Lab 4: deploy a web page by hand on your instance, bake it into an AMI, then launch a second server from that AMI with user data.



Time for Hands-On Practice!



Lab 4: Deploying a Web Application on EC2



What You'll Build:

-  Install Apache and deploy a page on `userN-web-1` by hand
-  Create a custom AMI from the configured server
-  Launch `userN-web-2` from the AMI in the other AZ with a user-data script — zero manual steps

Lunch follows



Ship It!



Module 2D

Storage — Amazon S3 and Amazon EBS



Amazon Simple Storage Service (S3)

Durable, virtually unlimited *object* storage accessed over HTTPS. Not a file system — you PUT and GET whole objects.

- **Bucket** — a container with a *globally unique* name, created in one Region (data never leaves it unless you copy it)
- **Object** — data (0 bytes – 5 TB) + metadata, identified by a **key** such as `images/2026/logo.png` (folders are a display convention)
- **11 nines durability** (99.999999999%) — replicated across ≥ 3 AZs automatically
- **Strong read-after-write consistency** for all operations
- Pay for storage used, requests, and data transfer *out*; unlimited scale

Common uses

- Backup and archive · data lakes · static website hosting · media storage · log storage · software distribution · disaster recovery

Features you will use in Lab 5

- **Versioning** — keep every version; recover from overwrites and deletes. Once enabled it can only be suspended
- **Static website hosting** — serve HTML/CSS/JS directly from the bucket
- **Block Public Access** — on by default; you must deliberately turn it off *and* add a bucket policy to publish
- **Bucket policies** (resource-based JSON) · **IAM policies** · ACLs (legacy) · **presigned URLs** (time-limited links)
- **Lifecycle rules** — transition objects to cheaper classes or expire them after N days
- **Encryption at rest** — SSE-S3 is on by default; SSE-KMS for your own keys
- **Event notifications** → Lambda, SQS, SNS
- **Replication** — same-Region or cross-Region copies



Buckets are Regional; names are global. "Recover an accidentally deleted object" → versioning. "Share a private object for 1 hour" → presigned URL.



S3 Storage Classes — Pay for the Access Pattern You Have

Class	Designed for	Retrieval	Relative cost
S3 Standard	Frequently accessed data	Milliseconds	\$\$\$
S3 Intelligent-Tiering	Unknown or changing access patterns — moves objects between tiers automatically	Milliseconds	\$\$ + small monitoring fee
S3 Standard-IA	Infrequent access, rapid when needed (backups); multi-AZ	Milliseconds; per-GB retrieval fee	\$\$
S3 One Zone-IA	Re-creatable infrequent data; single AZ	Milliseconds	\$
S3 Glacier Instant Retrieval	Archive accessed ~once a quarter	Milliseconds	\$
S3 Glacier Flexible Retrieval	Archive accessed 1–2× a year	Minutes to hours	¢
S3 Glacier Deep Archive	Compliance archives kept 7–10 years	12 – 48 hours	¢ (lowest)

Other storage services

- **Amazon EFS** — shared NFS file system for many Linux instances; grows automatically
- **Amazon FSx** — managed Windows File Server (SMB) and Lustre (HPC)
- **AWS Storage Gateway** — hybrid: on-prem file / volume / tape gateway backed by S3
- **AWS Snow Family** — physical devices to move petabytes when the network is too slow
- **AWS Backup** — central backup policies across EBS, RDS, DynamoDB, EFS ...



Exam tip: "Lowest cost, retrieval in hours is fine" → Glacier Deep Archive. "Unpredictable access" → Intelligent-Tiering. "Shared file system for many EC2 instances" → EFS. "Windows file shares" → FSx for Windows. "Move 80 TB with a slow link" → Snowball Edge.

Amazon Elastic Block Store (EBS)

Persistent block storage volumes for EC2 — the instance sees a hard drive. You saw one in Lab 3 as the root volume.

- Lives in **one AZ**; attach to an instance in the same AZ (one volume ↔ one instance, except io2 Multi-Attach)
- Persists independently of the instance — survives stop/start; detach and re-attach elsewhere
- Automatically replicated within its AZ
- Resize or change type *while in use*
- Billed for provisioned GB-month even when the instance is stopped — delete unused volumes (Lab 8)

Snapshots

- Point-in-time, incremental backups stored in S3
- Copy across AZs and Regions; create volumes or AMIs from them (Lab 4's AMI is an EBS snapshot + metadata)

Volume type	Use case
gp3 (general purpose SSD)	Default for boot volumes, dev/test, most workloads — 3,000 IOPS baseline, tune IOPS and throughput independently
io2 (provisioned IOPS SSD)	Critical databases needing sustained high IOPS and 99.999% durability
st1 (throughput HDD)	Big sequential workloads — logs, big data, data warehouses
sc1 (cold HDD)	Infrequently accessed data at the lowest cost

EBS vs S3 vs EFS vs instance store

	Type	Attached to
EBS	Block	One instance, one AZ
Instance store	Block (ephemeral)	The host; gone on stop
EFS	File (NFS)	Many instances, many AZs
S3	Object	Anything over HTTPS

Up next — Lab 5: create a versioned bucket, host a static website from it, and read it from your EC2 instance through the IAM role.



Time for Hands-On Practice!



Lab 5: Using S3 for Object Storage



What You'll Build:

-  A versioned bucket — upload, overwrite, and see the old version survive
-  A static website with a bucket policy (Block Public Access deliberately relaxed)
-  Read the bucket from your EC2 instance via the IAM role — and watch a write get denied



Store Everything!

Module 2C (continued)

Serverless Compute — AWS Lambda



AWS Lambda: Run Code Without Servers

Upload a function; AWS runs it in response to events and bills you per millisecond of execution. No instances to size, patch or scale.

- Languages: Python, Node.js, Java, .NET, Go, Ruby, custom runtimes, container images
- Scales automatically from zero to thousands of concurrent executions
- Max 15 minutes per invocation; memory 128 MB – 10 GB (CPU scales with memory)
- Permissions come from an **execution role** (IAM role Lambda assumes) — least privilege again
- Logs and metrics land in CloudWatch automatically
- Free tier: 1 million requests and 400,000 GB-seconds *per month, forever*

```
def lambda_handler(event, context):  
    name = event.get("name", "World")  
    return {"statusCode": 200,  
            "body": f"Hello, {name}!"}
```

Common triggers

- **S3** — object created → resize image, scan file
- **API Gateway / Function URL** — HTTP request → serverless API
- **EventBridge** — schedule (cron) or AWS events → nightly cleanup, react to instance state change
- **SQS / SNS / DynamoDB Streams / Kinesis** — process messages and records

```
user      --> API Gateway  --> Lambda --> DynamoDB  
upload    --> S3 event    --> Lambda --> S3 (thumbnail)  
schedule  --> EventBridge --> Lambda --> stop idle EC2
```

The serverless family

- **AWS Fargate** — serverless containers for ECS/EKS
- **Amazon API Gateway** — managed REST/HTTP/WebSocket APIs
- **Amazon SQS / SNS / EventBridge / Step Functions** — decoupling and orchestration
- **Amazon DynamoDB, Aurora Serverless** — serverless data stores



Exam tip: "Run code only when triggered, pay only for execution time, no server management" → Lambda. "Longer than 15 minutes" → EC2, ECS/Fargate or AWS Batch.

Up next — Lab 6: write and test a Python function, read its logs in CloudWatch, and give it a public Function URL.



Time for Hands-On Practice!



Lab 6: Implementing Serverless Functions with Lambda



What You'll Build:

- A Python Lambda function
- Test it and read the duration, memory and logs in CloudWatch
- A public Function URL — an HTTPS endpoint with no server, security group or VPC



Go Serverless!



Module 2E

Databases — Amazon RDS and Amazon DynamoDB



Amazon Relational Database Service (RDS)

Managed relational databases: AWS handles provisioning, patching, backups, replication and failover — you handle schema, queries and users.

- Engines: **MySQL, PostgreSQL, MariaDB, Oracle, SQL Server, Db2** and **Amazon Aurora**
- Runs on a DB instance class (e.g. `db.t3.micro`) inside your VPC — reached via a DNS endpoint, protected by a security group
- Automated backups (1–35 day retention, point-in-time restore) and manual snapshots
- **Multi-AZ** — synchronous standby in another AZ; automatic failover for *availability*
- **Read replicas** — asynchronous copies (same or cross-Region) for *read scaling*
- Encryption at rest with KMS; TLS in transit; IAM database authentication
- Scale compute by changing the class; storage autoscaling

Amazon Aurora

- AWS-built, MySQL- and PostgreSQL-compatible; up to 5x / 3x the throughput
- Storage auto-grows to 128 TB, replicated 6 ways across 3 AZs, continuous backup to S3
- Up to 15 read replicas; **Aurora Serverless** scales capacity automatically

Other database services

- **Amazon Redshift** — petabyte data warehouse for analytics (OLAP)
- **Amazon ElastiCache** — in-memory Redis / Memcached for sub-millisecond caching
- **Amazon DocumentDB** (MongoDB-compatible) · **Neptune** (graph) · **Timestream** · **QLDB**
- **AWS Database Migration Service** — migrate on-prem databases with minimal downtime (homogeneous or with Schema Conversion Tool)



Exam tip: "Improve availability" → Multi-AZ. "Offload read traffic" → read replica. "Data warehouse / BI queries" → Redshift. "Cache to reduce DB load" → ElastiCache.



Amazon DynamoDB — Serverless NoSQL

Fully managed key-value and document database with single-digit-millisecond latency at any scale. No servers, no patching, no capacity planning (on-demand mode).

- **Table** → **items** → **attributes**; schemaless except the primary key
- **Partition key** (required) spreads data across storage; optional **sort key** orders items within a partition — e.g. **Artist** + **SongTitle**
- Secondary indexes for other query patterns
- Capacity: **on-demand** (pay per request) or **provisioned** (RCUs/WCUs with auto scaling)
- Eventually consistent reads by default; strongly consistent on request
- Replicated across 3 AZs; **Global Tables** for multi-Region active-active
- Streams, TTL, point-in-time recovery, DAX (in-memory cache), encryption at rest

Relational vs NoSQL

Relational (RDS)	NoSQL (DynamoDB)
Fixed schema, normalized tables, joins	Flexible schema, denormalized, no joins
SQL; ACID transactions	API calls (PutItem, Query); transactions available
Scales up (bigger instance) + read replicas	Scales out horizontally, automatically
Complex queries and reporting	Known access patterns at massive scale — sessions, carts, IoT, gaming, profiles

```
{ "Artist": "Nina Simone",  
  "SongTitle": "Feeling Good",  
  "Year": 1965,  
  "Tags": ["jazz", "soul"] }
```

 **Exam tip:** "Serverless NoSQL, millisecond latency, any scale" → DynamoDB. "Existing MySQL app, minimal changes" → RDS/Aurora.

Up next — Lab 7: launch a MySQL RDS instance into your VPC (Part A — it provisions during the break), then connect from your EC2 instance, create a table, and find backups and Multi-AZ (Part B).



Time for Hands-On Practice!



Lab 7: Setting Up a Relational Database with RDS



What You'll Build:

- **Part A (before the break):** a DB subnet group and a MySQL db.t3.micro in `userN-vpc` with `userN-db-sg`
- **Part B (after the break):** connect from your EC2 instance — only the web security group can reach it
- Create a table, then find backups, Multi-AZ and instance class in the console

Part A now • Break • Part B after the break



Start the Database, Then Take a Break!



Module 3

AWS Global Infrastructure



Regions and Availability Zones

Regions

- A physical location containing multiple AZs: `us-east-1` N. Virginia, `us-east-2` Ohio, `eu-west-1` Ireland, `ap-southeast-1` Singapore ...
- 30+ Regions, fully independent — a failure or a maintenance event in one cannot affect another
- Data stays in the Region you choose unless *you* move it
- Not every service, instance type or feature is in every Region; prices differ
- Special Regions: AWS GovCloud (US), China (operated by local partners)

Choosing a Region

- **Compliance** — data residency and governance requirements
- **Proximity** — latency to your users
- **Feature availability** — new services land in a few Regions first
- **Pricing** — the same instance costs different amounts in different Regions

Availability Zones

- One or more discrete data centers with redundant power, networking and cooling
- Every Region has **at least 3 AZs**; physically separated (miles apart, distinct flood plains and power grids) but joined by low-latency private fiber
- Designed so a fire, flood or power loss in one AZ does not affect the others
- Services like S3, DynamoDB and Lambda are Regional and use multiple AZs for you; EC2, EBS, subnets and RDS instances are *zonal* — **you** spread them across AZs

```
Region us-east-2
|- AZ us-east-2a - data center(s) - your subnet 1, web-1, RDS primary
|- AZ us-east-2b - data center(s) - your subnet 2, web-2, RDS standby
+- AZ us-east-2c - data center(s)
```



Exam tip: Multi-AZ = high availability within a Region. Multi-Region = disaster recovery and global latency. "AZ" letters map differently per account, so `us-east-2a` is not the same building for everyone.



Edge Locations and AWS Global Accelerator

Edge locations / Points of Presence

- **600+ sites** in hundreds of cities — far more than Regions, and much closer to users
- Home of **Amazon CloudFront** (content caching), **Route 53** (DNS resolution), **AWS WAF / Shield** (edge protection) and **Lambda@Edge / CloudFront Functions**
- Regional edge caches sit between edge locations and your origin for less-popular content
- You do not launch servers in edge locations; they serve and protect traffic

Extending the Region

- **AWS Local Zones** — compute and storage in large metro areas (single-digit ms for gaming, media, AR/VR)
- **AWS Wavelength** — inside telecom 5G networks
- **AWS Outposts** — AWS racks and services in *your* data center

AWS Global Accelerator

- Gives your application **two static anycast IP addresses** advertised from every edge location
- User traffic enters the AWS global backbone at the nearest edge, avoiding congested public internet paths
- Routes to the nearest *healthy* endpoint (ALB, NLB, EC2, Elastic IP) across Regions; instant failover
- Works for TCP and UDP, not just HTTP — gaming, VoIP, IoT

	CloudFront	Global Accelerator
What it does	Caches <i>content</i> at the edge	Accelerates <i>connections</i> to your endpoints
Best for	Static and streaming content, HTTP(S)	Dynamic apps, non-HTTP, static IP needs, multi-Region failover
IPs	Changing edge IPs via DNS	2 fixed anycast IPs



Exam tip: "Static IP addresses for a global TCP/UDP application" → Global Accelerator. "Cache images and video near users" → CloudFront. Both use edge locations.



Module 4

Billing and Cost Optimization



Understanding AWS Pricing

Three fundamental drivers

- **Compute** — per second/hour an instance runs, per ms a function executes
- **Storage** — per GB-month stored (plus requests for S3)
- **Data transfer out** — per GB leaving AWS to the internet. Inbound is free; transfer between AZs and Regions is charged

Principles

- Pay-as-you-go — no long-term contracts by default, no termination fees
- Pay less when you reserve, pay less by using more (tiered S3 pricing), pay less as AWS grows
- **Free Tier:** *Always free* (1M Lambda requests/month, 25 GB DynamoDB), *free trials*, and a credit-based plan for accounts created after July 2025 (older accounts: 12-months-free, e.g. 750 h t2/t3.micro)

AWS Pricing Calculator (calculator.aws) — estimate a workload before you build; save and share estimates (Lab 8).

EC2 purchasing options

Option	Commitment	Savings	Use for
On-Demand	None; per second	—	Dev/test, unpredictable, short-term
Savings Plans	\$/hour for 1 or 3 years (Compute or EC2 Instance)	Up to 72%	Steady usage; flexible across types, Regions, Fargate, Lambda
Reserved Instances	Specific type/Region, 1 or 3 years; no / partial / all upfront	Up to 72%	Steady, predictable workloads; also RDS, ElastiCache, Redshift
Spot Instances	None — AWS can reclaim with 2-minute warning	Up to 90%	Fault-tolerant batch, CI, big data, rendering
Dedicated Hosts / Instances	Physical server for you	—	Licensing (BYOL), compliance requiring single tenancy
Capacity Reservations	Reserve capacity in an AZ, no discount	—	Guarantee capacity for an event



Exam tip: "Cheapest for interruptible work" → Spot. "Steady 24×7 for 3 years, most flexible" → Compute Savings Plan. "Server-bound software licenses" → Dedicated Host.



Billing and Cost Management Tools

Billing & Cost Management console

Invoices, payment methods, Free Tier usage, credits, tax settings.

AWS Cost Explorer

Visualize and analyze spend and usage over 13 months by service, account, tag or Region; forecast 12 months; RI/Savings Plan recommendations.

AWS Budgets

Set cost, usage, RI or Savings Plan budgets; get alerts (email/SNS) at thresholds; optional automated actions.

AWS Cost Anomaly Detection

ML flags unusual spend and notifies you.

Cost & Usage Report (CUR)

The most granular billing data, delivered to S3 for Athena/QuickSight analysis.

Cost allocation tags

Activate tags like **CostCenter** / **Project** so Cost Explorer can slice spend by team.

AWS Organizations

Manage many accounts; **consolidated billing** = one bill, shared volume discounts and reservations; Service Control Policies for guardrails.

AWS Control Tower

Sets up a secure multi-account landing zone on top of Organizations.

AWS Trusted Advisor

Checks for idle load balancers, under-utilized instances, unattached EBS volumes, RI opportunities (full checks need Business/Enterprise support).

AWS Compute Optimizer

ML recommendations to rightsize EC2, ASG, EBS and Lambda based on 14+ days of metrics.

AWS Pricing Calculator

Pre-deployment estimates.

Migration Evaluator / TCO

Compare on-premises total cost of ownership with AWS.



Exam tip: "Alert me when forecast spend exceeds \$X" → Budgets. "Which service cost the most last month?" → Cost Explorer. "One bill for 20 accounts" → Organizations consolidated billing. "Detailed line items in S3" → CUR.



Cost Optimization Strategies

1. Rightsize

- Match instance type and size to actual CPU/memory/network utilization — most fleets are over-provisioned
- Use CloudWatch metrics, Compute Optimizer and Trusted Advisor; downsize in steps (t3.large → t3.medium) or change family (Graviton ~20% cheaper)
- Rightsize storage too: gp2 → gp3, delete unattached volumes and old snapshots, S3 lifecycle to cheaper classes

2. Commit for steady state

- Cover the baseline with Savings Plans / Reserved Instances; keep the peaks On-Demand or Spot

3. Turn things off

- Schedule dev/test instances to stop nights and weekends (Instance Scheduler, EventBridge + Lambda) — ~65% savings
- Delete idle load balancers, NAT gateways, Elastic IPs not attached (they cost money!), forgotten RDS instances

4. Auto Scaling — pay for demand, not for peak

- **EC2 Auto Scaling group**: launch template + min/desired/max + scaling policies; replaces unhealthy instances; spreads across AZs
- **Target tracking** ("keep average CPU at 50%") · **step / simple** (CloudWatch alarm thresholds) · **scheduled** (known peaks) · **predictive** (ML forecast)
- Attach to a load balancer so new instances receive traffic automatically
- Same idea elsewhere: DynamoDB auto scaling, Aurora Serverless, Lambda (scales itself), Application Auto Scaling for ECS

```
CloudWatch CPU > 70% --> alarm --> ASG adds an instance --> ALB routes traffic to it
CloudWatch CPU < 30% --> alarm --> ASG removes an instance
                        min 1 --- desired 2 --- max 4
```

5. Use managed and serverless services

- Lambda, Fargate, DynamoDB, S3 — no idle capacity to pay for; less operations time



"Handle variable traffic cost-effectively" → Auto Scaling + ELB.
"Reduce cost of a consistently under-utilized instance" → rightsize.

Up next — Lab 8: compare On-Demand vs Savings Plan pricing, set a budget alert, rightsize an instance — then clean up everything you built today.



Time for Hands-On Practice!



Lab 8: Implementing Cost Optimization Strategies



What You'll Do:

-  Price your web servers On-Demand vs Savings Plan in the Pricing Calculator
-  Create a Budget with an email alert
-  Rightsize `userN-web-2` to a smaller instance type
-  **Clean up** everything from Labs 1–7 so nothing is billed overnight

Cleanup checklist is at the end of the lab guide



Spend Smart!



Module 5

Best Practices and Troubleshooting



Building Secure and Reliable Applications on AWS



Security best practices

- MFA on root and every user; no root access keys; least-privilege IAM; roles for machines
- Security groups scoped to exact ports and sources; SSH via Session Manager, not 0.0.0.0/0
- Encrypt everything at rest (KMS) and in transit (TLS/ACM); keep S3 Block Public Access on unless you mean it
- Secrets in Secrets Manager or Parameter Store — never in code or user data
- Turn on CloudTrail (all Regions), GuardDuty, Config; review Trusted Advisor security checks and IAM Access Analyzer
- Patch on a schedule (Systems Manager Patch Manager); tag every resource



Reliability best practices

- **Design for failure** — assume any instance, AZ or dependency can fail; no single points of failure
- **Multi-AZ everything** — subnets, Auto Scaling groups, ELB, RDS Multi-AZ; multi-Region for DR when the business needs it
- **Stateless servers** — sessions in DynamoDB / ElastiCache, files in S3 / EFS, so instances are disposable
- **Decouple** with SQS, SNS, EventBridge so a slow component does not take down the rest
- **Automate** — infrastructure as code (CloudFormation / CDK), immutable AMIs, CI/CD; humans make typos at 2 a.m.
- **Monitor and alarm** — CloudWatch metrics, logs and alarms; test backups and recovery (RTO / RPO) before you need them

```
Route 53 --> CloudFront + WAF --> ALB (2 AZs) --> Auto Scaling group (stateless web tier) --> RDS Multi-AZ / DynamoDB
                                     |
                                     +--> ElastiCache
                                     +--> S3 (static assets, logs)
                                     CloudWatch + CloudTrail + GuardDuty watch it all
```



Common Troubleshooting Techniques

"I can't reach my instance / web page"

1. Is it **running** and passing both status checks?
2. Does it have a **public IP** (or Elastic IP)?
3. Is the subnet's **route table** sending `0.0.0.0/0` to an IGW?
4. **Security group**: inbound rule for the port from your source?
5. **Network ACL**: inbound *and* outbound allowed, including ephemeral ports 1024–65535?
6. Is the **service listening** on the instance (`systemctl status httpd`, OS firewall)?
7. Read the **system log** / user-data output: `/var/log/cloud-init-output.log`

"Access Denied"

- Which identity am I? (`aws sts get-caller-identity`)
- Is there an explicit Deny anywhere (SCP, permissions boundary, bucket policy)? Deny always wins
- S3: Block Public Access, bucket policy *and* IAM policy must all agree
- Use the **IAM Policy Simulator** and **CloudTrail** (`errorCode: AccessDenied`)

Your toolbox

Tool	Answers
Amazon CloudWatch	Metrics (CPU, latency, errors), logs, alarms, dashboards — "what is the system doing?"
AWS CloudTrail	Who called which API, when, from where — "who changed this?"
AWS Config	Configuration history and compliance — "what did it look like yesterday?"
VPC Flow Logs	Accepted / rejected traffic per interface — "is the packet being dropped?"
VPC Reachability Analyzer	Path analysis between two resources — "why can't A reach B?"
AWS X-Ray	Distributed tracing across services — "which call is slow?"
AWS Health Dashboard	Is AWS itself having an issue affecting my account?
AWS Support & re:Post	Cases per your support plan; community Q&A; AWS Trusted Advisor

Support plans

- **Basic** (free) · **Developer** (business hours email) · **Business** (24×7 phone/chat, full Trusted Advisor, 1-hour response for production down) · **Enterprise On-Ramp** · **Enterprise** (15-minute critical response, Technical Account Manager, Concierge)



The AWS Well-Architected Framework

Best practices and questions developed by AWS architects to evaluate workloads. It does not prescribe implementations — it helps you **assess, improve and understand the business impact** of design decisions. Run a review with the free **AWS Well-Architected Tool** in the console.



Operational Excellence

Operations as code, small reversible changes, refine procedures often, anticipate failure, learn from every event.



Security

Strong identity foundation, traceability, security at all layers, least privilege, protect data in transit and at rest, automate, prepare for incidents.



Reliability

Recover automatically, test recovery, scale horizontally, stop guessing capacity, manage change through automation.



Performance Efficiency

Democratize advanced technologies, go global in minutes, use serverless, experiment often, mechanical sympathy.



Cost Optimization

Practice cloud financial management, adopt a consumption model, measure efficiency, stop undifferentiated heavy lifting, attribute spend.



Sustainability

Understand your impact, set targets, maximize utilization, adopt efficient hardware and managed services, reduce downstream impact.



Exam tip: Six pillars (Sustainability was added in 2021). Every lab today mapped to at least one: least privilege (Security), Multi-AZ subnets (Reliability), rightsizing (Cost Optimization), AMLs and user data (Operational Excellence).



Preparing for the AWS Certified Cloud Practitioner Exam (CLF-C02)

Exam facts

- **65 questions** (50 scored + 15 unscored), **90 minutes**, passing score **700 / 1000**, USD 100
- Multiple choice (1 of 4) and multiple response (2+ of 5); no penalty for guessing — **answer everything**
- Testing center or online proctored; valid 3 years
- Flag questions and come back; ~80 seconds per question

Domains

Domain	Weight
1. Cloud Concepts	24%
2. Security and Compliance	30%
3. Cloud Technology and Services	34%
4. Billing, Pricing and Support	12%

How to study

- **AWS Skill Builder** — free "Cloud Practitioner Essentials" course and the official practice question set
- Read the **CLF-C02 exam guide** and the whitepapers: *Overview of Amazon Web Services*, *How AWS Pricing Works*, *Well-Architected Framework*
- Skim each core service's **FAQ** — if you can answer the FAQ, you can answer the exam
- Take a full-length practice exam under time; review every wrong answer
- Keep doing what you did today: build it, break it, fix it

Answering technique

- Look for the keyword ("cache", "DDoS", "single bill", "interruptible") — it usually names the service
- Prefer the *managed* service, the *least-privilege* option and the *Multi-AZ* design
- Eliminate answers that violate the Shared Responsibility Model

Recommended next step: schedule the exam within 2–4 weeks while today is fresh.

? Practice Questions — Round 1

1. A company wants to run a nightly batch job that can be interrupted and restarted without harm. Which EC2 purchasing option is the *most* cost-effective?

- A. On-Demand
- B. Reserved Instances
- C. Spot Instances
- D. Dedicated Hosts

C. Spot Instances offer up to 90% off for interruptible workloads.

2. Under the Shared Responsibility Model, which task is the customer's responsibility?

- A. Replacing failed disks in the data center
- B. Patching the guest operating system on EC2
- C. Maintaining the hypervisor
- D. Physical access control to facilities

B. Everything *in* the instance is yours; the facility, hardware and hypervisor are AWS's.

3. Which AWS service lets an application running on EC2 access S3 *without* storing credentials on the instance?

- A. IAM user with access keys
- B. IAM role attached to the instance
- C. AWS KMS
- D. Amazon Cognito

B. Roles provide temporary, automatically rotated credentials — exactly what you used in Lab 5.

4. A firewall that is *stateless* and operates at the *subnet* level is a:

- A. Security group
- B. AWS WAF web ACL
- C. Network ACL
- D. AWS Shield

C. Network ACLs are stateless and subnet-scoped; security groups are stateful and instance-scoped.

? Practice Questions — Round 2

5. Which service provides a single consolidated bill and shared volume discounts across many AWS accounts?

- A. AWS Budgets
- B. AWS Organizations
- C. AWS Cost Explorer
- D. AWS Control Tower

B. Organizations' consolidated billing. Control Tower uses Organizations to build a landing zone.

6. A company needs to archive compliance records for 10 years and can wait up to 12 hours to retrieve them. Which storage class is the lowest cost?

- A. S3 Standard-IA
- B. S3 Intelligent-Tiering
- C. S3 Glacier Instant Retrieval
- D. S3 Glacier Deep Archive

D. Deep Archive — lowest cost, 12–48 hour retrieval.

7. Which TWO actions improve the availability of an application running on EC2? (Choose two)

- A. Deploy instances in multiple Availability Zones behind an Elastic Load Balancer
- B. Use a larger instance type
- C. Use an Auto Scaling group to replace unhealthy instances
- D. Enable S3 versioning
- E. Purchase Reserved Instances

A and C. Multi-AZ + ELB and Auto Scaling self-healing are the availability levers; B, D, E address other concerns.

8. Which service should a company use to receive an alert when its monthly AWS spend is forecast to exceed \$5,000?

- A. AWS Cost Explorer
- B. AWS Trusted Advisor
- C. AWS Budgets
- D. Amazon CloudWatch

C. Budgets sends threshold alerts on actual or forecasted cost — you created one in Lab 8.



Congratulations!

You built a real AWS environment today

What you built

- An IAM group, user and EC2 role with least privilege
- A VPC with two subnets, security groups and a network ACL rule
- Two EC2 web servers, a custom AMI and an automated deployment
- A versioned S3 bucket serving a static website
- A Lambda function with an HTTPS endpoint
- A MySQL database on RDS reached only through the web security group
- A budget, a pricing comparison and a rightsized instance

Before you leave

- Confirm your Lab 8 cleanup checklist is complete
- Book the CLF-C02 exam
- AWS Skill Builder → Cloud Practitioner Essentials + practice question set
- Create your own free-tier account and rebuild today's labs from memory

Thank you! Questions, feedback and war stories welcome.